



Unifor

UNIVERSIDADE DE FORTALEZA

CENTRO DE CIÊNCIAS TECNOLÓGICAS

CURSO: CIÊNCIA DA COMPUTAÇÃO

Disciplina	Resolução de Problemas com Grafos
Tema	Fluxo máximo em redes e modelagem por capacidades
Problema	UVA 259 - Software Allocation
Professor	Prof. Me. Ricardo Carubbi
Integrantes	Átila Silvio; Lucas Barroso; Victor Menezes Do Vale.

1. Resumo do problema

O problema Software Allocation trata da alocação de aplicações em computadores de um centro de processamento.

Existem 10 computadores, numerados de 0 a 9. Cada computador só pode executar uma aplicação por dia, pois os computadores não são multitarefa. Também existem aplicações identificadas por letras maiúsculas de A a Z.

Para cada aplicação, a entrada informa a letra da aplicação, a quantidade de usuários que solicitaram essa aplicação e em quais computadores ela pode ser executada.

Quando uma aplicação é solicitada por mais de um usuário, significa que precisamos alocar mais de uma cópia independente dessa aplicação, cada uma em um computador diferente.

O objetivo é verificar se é possível alocar todas as aplicações solicitadas respeitando as restrições. Se for possível, devemos imprimir uma distribuição dos programas nos computadores. Se não for possível, devemos imprimir !.

2. Interpretação da entrada e da saída

A entrada é composta por vários casos, chamados de dias. Cada dia possui uma ou mais linhas de descrição de aplicações. Os dias são separados por uma linha em branco, e a entrada termina no fim do arquivo.

Cada linha segue o formato:

```
A4 01234;
```

Nesse exemplo:

- A é a aplicação;
- 4 indica que 4 usuários solicitaram a aplicação A;
- 01234 indica que a aplicação A pode rodar nos computadores 0, 1, 2, 3 e 4;
- ; encerra a linha.

A saída deve ter uma linha para cada dia. Se a alocação for possível, a saída deve conter exatamente 10 caracteres, um para cada computador de 0 a 9.

Exemplo de saída possível:

```
AAAA_QPPPP
```

Essa saída indica que os computadores 0, 1, 2 e 3 receberam a aplicação A, o computador 4 ficou livre, indicado por _, o computador 5 recebeu Q e os computadores 6, 7, 8 e 9 receberam P.

Se não for possível alocar todas as aplicações, a saída deve ser:

```
!
```

3. Modelagem da rede de fluxo

O problema pode ser modelado como uma rede de fluxo máximo. A ideia principal é transformar cada cópia de aplicação solicitada em uma unidade de fluxo. Se conseguirmos enviar todo o fluxo da origem até o sorvedouro, então conseguimos alocar todas as aplicações.

3.1 Vértices da rede

Elemento	Interpretação no problema
Origem s	Representa o início da distribuição das aplicações.
Aplicações A-Z	Cada vértice representa uma aplicação solicitada no dia.
Computadores 0-9	Cada vértice representa uma máquina disponível para receber no máximo uma aplicação.
Sorvedouro t	Representa o fim da alocação. Quando uma unidade de fluxo chega em t, uma cópia de aplicação foi alocada em algum computador.

3.2 Arestas e capacidades

A rede é formada por três grupos principais de arestas:

Elemento	Interpretação no problema
s -> aplicação	Capacidade igual ao número de usuários que solicitaram aquela aplicação. Exemplo: A4 gera s -> A com capacidade 4.
aplicação -> computador	Capacidade 1, criada apenas quando aquela aplicação pode rodar naquele computador.
computador -> t	Capacidade 1, pois cada computador só pode executar uma aplicação por dia.

Exemplo para a linha A4 01234,;

s -> A, capacidade 4

A -> 0, capacidade 1

A -> 1, capacidade 1

A -> 2, capacidade 1

A -> 3, capacidade 1

A -> 4, capacidade 1

3.3 Justificativa das capacidades

A capacidade da aresta s -> aplicação limita quantas cópias daquela aplicação precisam sair da origem. Se a aplicação A foi solicitada por 4 usuários, no máximo 4 unidades de fluxo podem sair de s para A.

As arestas aplicação -> computador representam compatibilidade. Só criamos essas arestas quando o enunciado permite que aquela aplicação rode naquele computador.

As arestas computador -> t têm capacidade 1, garantindo que cada computador receba no máximo uma aplicação. Portanto, se o fluxo máximo for igual ao total de aplicações solicitadas, existe uma alocação válida. Caso contrário, alguma aplicação não conseguiu ser alocada e a resposta deve ser !.

4. Justificativa da escolha entre Ford-Fulkerson e Edmonds-Karp

O algoritmo escolhido será o Edmonds-Karp, que é uma variação do Ford-Fulkerson.

O Ford-Fulkerson funciona aumentando o fluxo aos poucos por meio de caminhos aumentantes no grafo residual. O Edmonds-Karp segue a mesma lógica, mas usa BFS para escolher o caminho aumentante com menor número de arestas.

Essa escolha é adequada para o problema porque:

- a rede do problema é pequena;
- existem no máximo 10 computadores;
- o fluxo máximo nunca passa de 10, pois há apenas 10 computadores disponíveis;
- a BFS torna a escolha dos caminhos aumentantes mais previsível;
- a implementação fica mais clara para explicar e verificar manualmente.

A complexidade teórica do Edmonds-Karp é:

$$O(V * E^2)$$

Neste problema, esse custo é tranquilo, pois o número de vértices e arestas é pequeno. A memória é dominada pela lista de adjacência da rede residual, que armazena as arestas diretas e reversas.

5. Instância pequena

Para demonstrar o funcionamento manual do algoritmo, usaremos uma instância criada pelo grupo:

```
A1 01;  
B1 0;
```

Interpretação da instância:

- a aplicação A precisa de 1 computador e pode rodar nos computadores 0 e 1;
- a aplicação B precisa de 1 computador e só pode rodar no computador 0.

O total de aplicações solicitadas é:

$$1 + 1 = 2$$

Logo, para a alocação ser possível, o fluxo máximo precisa ser igual a 2.

5.1 Rede de fluxo da instância

A rede terá as seguintes arestas:

```
s -> A, capacidade 1  
s -> B, capacidade 1  
  
A -> 0, capacidade 1  
A -> 1, capacidade 1  
B -> 0, capacidade 1  
  
0 -> t, capacidade 1  
1 -> t, capacidade 1
```

Os caminhos possíveis inicialmente são:

```
s -> A -> 0 -> t  
s -> A -> 1 -> t  
s -> B -> 0 -> t
```

Essa instância é útil porque mostra uma situação em que o algoritmo precisa usar uma aresta reversa do grafo residual para corrigir uma escolha anterior.

6. Execução manual passo a passo

6.1 Estado inicial

Todos os fluxos começam com valor 0.

```
Fluxo total inicial = 0
```

6.2 Primeiro caminho aumentante

Considerando que a BFS encontra primeiro o caminho:

```
s -> A -> 0 -> t
```

As capacidades disponíveis são:

```
s -> A = 1  
A -> 0 = 1  
0 -> t = 1
```

O gargalo do caminho é:

```
min(1, 1, 1) = 1
```

Aumentamos o fluxo em 1 unidade nesse caminho. O novo fluxo total passa a ser 1. Nesse momento, a aplicação A fica alocada no computador 0.

Porém, isso cria um problema: a aplicação B só pode usar o computador 0, que já está ocupado por A.

6.3 Grafo residual após o primeiro aumento

Depois do primeiro caminho, a aresta A -> 0 fica saturada. No grafo residual, aparece uma aresta reversa:

```
0 -> A
```

Essa aresta reversa representa a possibilidade de desfazer a escolha anterior, ou seja, tirar A do computador 0 e realocá-la para outro computador.

6.4 Segundo caminho aumentante

Agora a BFS pode encontrar o seguinte caminho no grafo residual:

```
s -> B -> 0 -> A -> 1 -> t
```

Esse caminho usa a aresta reversa 0 -> A. O significado é:

1. mandar B para o computador 0;
2. desfazer a alocação anterior de A no computador 0;
3. realocar A no computador 1.

As capacidades residuais nesse caminho são todas iguais a 1, então o gargalo é 1. Aumentamos o fluxo em mais 1 unidade.

Novo fluxo total = 2

7. Verificação da resposta final

Ao final, a alocação obtida é:

```
computador 0 -> B
computador 1 -> A
```

Os demais computadores ficam livres. Como o fluxo máximo encontrado é 2 e o total de aplicações solicitadas também é 2, a alocação é possível.

7.1 Recuperação da resposta a partir do fluxo

Depois de calcular o fluxo máximo, verificamos as arestas que saem das aplicações para os computadores. Se uma aresta aplicação -> computador ficou com fluxo 1, isso significa que aquela aplicação foi alocada naquele computador.

Para montar a saída:

1. começamos com uma string de 10 posições preenchida com o caractere _;
2. para cada computador, verificamos se existe alguma aplicação enviada para ele;
3. se existir, colocamos a letra da aplicação na posição correspondente ao número do computador;
4. se não existir, mantemos o caractere _.

Se o fluxo máximo for menor que o total solicitado, imprimimos !.

7.2 Resposta final da instância pequena

Na instância pequena, o computador 0 recebeu B, o computador 1 recebeu A e os computadores de 2 a 9 ficaram livres. Portanto, a saída é:

```
BA_____
```

A string acima é composta por B, A e oito caracteres _ para indicar os computadores livres.

Essa resposta respeita todas as restrições:

- A foi alocada no computador 1, onde pode rodar;
- B foi alocada no computador 0, único computador onde pode rodar;
- nenhum computador recebeu mais de uma aplicação;
- todas as solicitações foram atendidas.